

LogPulse AI - Incident Report

Generated : 2026-05-29 17:29:33 UTC

Trace ID : 17f5d5c2-4cbb-4c38-a7a3-c54392c7cd07

1 - Incident Summary

Incident ID : 17f5d5c2
Category : Database
Severity : SeverityLevel.FATAL
Source : PostgreSQL
Cache : MISS

2 - Raw Log

[2026-05-26 14:32:01] ERROR [postgres-db-01] pid=1234 192.168.1.10:5432 FATAL: remaining connection slots are reserved for non-replication superuser

3 - Classification

Category : Database
Severity : SeverityLevel.FATAL
Summary : Connection pool exhausted on postgres-db-01
Source : PostgreSQL

4 - Diagnosis

Root Cause : Based on the matched playbook 'Database Connection Refused/Timed Out Post Node Maintenance', the likely root cause is a Database incident matching the pattern: Connection pool exhausted on postgres-db-01.
Confidence : 51%
Reasoning : Gemini unavailable; diagnosis derived from playbook 'Database Connection Refused/Timed Out Post Node Maintenance' (RAG confidence=0.5121).
Source Tier : playbook_fallback
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

Gemini unavailable; remediation derived from playbook 'Database Connection Refused/Timed Out Post Node Maintenance' (RAG confidence=0.5121). Review the playbook steps and adapt commands to the live environment before execution.

Steps:

- 1. Identify the Kubernetes node currently hosting the database pod using `kubectl get pod <db-pod-name> -o wide`.
- 2. SSH into the identified node.
- 3. Check the status of the `kube-proxy` service on the node: `sudo systemctl status kube-proxy.service`.
- 4. Review `kube-proxy` logs on the node for errors or warnings: `sudo journalctl -u kube-proxy.service --since '1 hour ago`.
- 5. Inspect `iptables` rules: `sudo iptables -L -n -v`. Look for missing or incorrect rules related to Kubernetes services and the database Service IP.
- 6. If `kube-proxy` is stopped or unhealthy, restart it: `sudo systemctl restart kube-proxy.service`.
- 7. If the issue persists after `kube-proxy` is confirmed healthy, consider restarting the database pod to force `kube-proxy` to re-establish rules specifically for its Service.

Commands:

```
$ kubectl get pod <db-pod-name> -o wide
$ SSH into the identified node.
$ iptables
$ kubectl rollout restart statefulset/northwind-db
```

6 - Risk Assessment

Risk Level : **RISKLEVEL.WARNING**

Safe Commands (3):

- [OK] kubectl get pod <db-pod-name> -o wide
- [OK] SSH into the identified node.
- [OK] iptables

Flagged Commands (1):

- [!] kubectl rollout restart statefulset/northwind-db [kubectl\s+rollout\s+restart]

7 - Operator Decision

Decision : OPERATORDECISION.APPROVED

Executed At : 2026-05-29T14:29:32.525560+00:00

Sandbox Results:

Command : kubectl get pod <db-pod-name> -o wide

Exit : 0

Time : 0ms

Output:

```
[auto-approve] simulated: kubectl get pod <db-pod-name> -o wide
```

Command : SSH into the identified node.

Exit : 0

Time : 0ms

Output:

```
[auto-approve] simulated: SSH into the identified node.
```

Command : iptables
Exit : 0
Time : 0ms

Output:

[auto-approve] simulated: iptables

Command : kubectl rollout restart statefulset/northwind-db
Exit : 0
Time : 0ms

Output:

[auto-approve] simulated: kubectl rollout restart statefulset/northwind-db

8 - Historical Context

Times Seen : 1

Past Outcomes:

- approved_executed on 2026-05-29 14:09:28.588744+00:00

9 - Incident Timeline

[14:29:32] WORKFLOW_COMPLETED -> Completed in 1400ms